

Lecture 4

Recap & Notes:

Why can testing not show the absence of defects?: since we cannot test the software exhaustively we use test cases of what we think might cause a failure to the system. The testing process only shows the defects. It never guarantees that our product is defect free.

Quiz – Answered:

1. What phase in the testing process determines “WHAT to test”? -> test analysis
2. The test concept is created during the following phase → test planning
3. Test design answers the question → how to test
4. During test analysis, we →) analyze the test basis and check for ambiguities
5. A test object is → the work result to a test

When to stop testing?:

When the test objectives have been achieved!: meaning the exit criteria has been met.
Not when the time is up, unless we are out of money.

Based on coverage: stop testing when everything that we have planned to test has been tested, features, components, platforms (IOS, Windows, Andriod), quality attributes (security, load testing, reliability)

Based on quality: when the product achieved sufficient quality based on the test results.

Severity-based testing: Major defects have been fixed and retested, and only minor issues remain.

Based on progress: Stop testing when all planned and necessary test activities are complete, including retests and prioritized areas.

coverage is like a checklist while progress is actually doing what's in this checklist

Q: Assuming you are a project's test manager. What would be your professional reasons for not being willing to stop an ongoing test? Choose two criteria that might not have been met yet.

A: based on coverage and based on progress we need to make sure that everything is covered and everything is done (progress) since quality is subjective and be always improved.

Q: Assuming you are a project's test manager. What would be your professional reasons for accepting a software after system test?

A: As a test manager, I would accept the software after system testing if all exit criteria are met (coverage complete), all critical defects are fixed (quality acceptable), all planned tests and retests are executed (progress completed), risks are acceptable, and stakeholders approve the release.

Why Test Analysis is Important:

Evaluation of Test “Input”

- Test analysis examines the test basis (requirements, specifications, design documents) to determine what should be tested.
- Example: Reviewing a login requirement to understand valid/invalid inputs, error messages, and workflows.

Identifying Known Defects

- During analysis, inconsistencies, inaccuracies, contradictions, or missing information in the requirements can be found before test execution, which saves time and effort.
- Example: Two requirements contradict each other regarding password rules. Detecting this early avoids wasted testing later.

Identification of Feature Set

- Test analysis helps clearly identify the features and functionalities that need to be tested, ensuring proper coverage.
- Example: From a mobile app spec, testers derive that login, profile, notifications, and payment features are all in scope.

What can be used as test basis?:

- 1) Requirements specification, user stories, functional/non-functional requirements, use cases, personas.
- 2) Design documentation, e.g. UML diagrams, architecture descriptions, etc:

Architecture descriptions, UML diagrams, database schemas, flowcharts.

Example: A sequence diagram can show the steps for a payment transaction, which can be translated into test cases.

- 3) Sometimes the implementation (the code) can be a test basis to drive test cases from it can show some possible defects.
- 4) Risk analysis reports, error reports: older defects that was found in similar modules can be used to drive a test case

- 5) User documentation: Manuals, help guides, tutorials that describe how users are expected to interact with the system. These while reading them can lead to finding a possible defect and thus writing a test case for it.

Test Resources:

Test Resources: All artifacts created during the testing process that are required to plan, design, or execute tests.

“Artifacts created during the testing process”

These are **documents, tools, or materials** produced specifically to support testing.

Includes: Documents, scripts, input data, expected results, procedures for setting up and cleaning up test data, files, databases, environments and other additional software and utilities used for testing.

Test Process:

Test activities: These are the specific tasks done during testing. planning tests, test analysis, designing test, implementing test, executing tests, logging defects, reporting results.

Independent process: Testing is a separate process from development to check quality.

Closely interlinked with software development: Even though testing is independent, it depends on development. Timing, inputs, and scope of tests are closely related to the development process.

Necessary: fine granular execution plan for tests in each test stage: Each test stage (unit, integration, system, acceptance) needs a detailed plan. This plan includes what to test, how to test it, with which data, and when.

Specific software testing process depends on many factors: There's no one-size-fits-all process. Factors affecting it include project size, team, risk, deadlines, technology, and company policies.

Each company implements its testing strategy differently

Test Process in Context:

How does the test process depend on context?: Small vs Big app

A smaller app might need informal testing with less testing cases and unit testing while a bigger app will need formal planning with planning process (test planning, test analyzing, ..)

Risk level

High-risk system (e.g., banking software): More thorough testing, more test levels, stricter exit criteria.

Low-risk system (e.g., internal tool for a small team): Less extensive testing may be acceptable.

- **SDLC style:** Agile makes the testing happen while the development is taking place while waterfall SDLC waits for the development to end so it can start testing
- **Possible test levels and test types:** Decide which type of test we need based on functional and non-functional requirements. For instance, testing the website for load when you expect high traffic, other app would need acceptance test since it uses a third party software.
- **Product and project risks:** meaning testing depth depends heavily on how critical or sensitive the product is. A bank app needs way more testing than a game for instance.
- **Business area:** The domain which the product is in plays a huge role, a device that operates a surgery need much more testing than a software that handles drinks,
- **Business restrictions:** The restrictions within the company developing the product itself, this maybe budget, time, laws, corporate identity.
- **Guidelines and compliance with the laws**

Traceability between test basis and test work:

What does “traceability between test basis and test work results” mean?:

Establishing relationships between each and every element of the test basis and the results of the test activities. Meaning linking the test basis (requirement) to a test case and link both of them to a test result.

Traceability: The degree to which a relationship can be established between two or more products of the development process.... Traceability means linking requirements or specifications (test basis) to the tests that check them and the results of those tests. It helps ensure that every requirement is tested and you can see which ones passed or failed.

Through traceability we check whether each requirement is covered by test conditions and test cases, whether each test condition has test cases, which test cases have been executed and their results, and whether the requirement passed all its associated test cases.

Why is it important: it ensures coverage, links results to requirements, support test control and testability. Test control is impossible without bidirectional traceability.

Benefits:

1. **Evaluate test coverage**
 - See if all requirements are properly tested.
2. **Change impact analysis**
 - If a requirement changes, know **which test cases are affected** and update the test cases.
3. **Improve test reporting**
 - Makes progress and completion reports more measurable. For example, 80 test cases of 100 were executed with 20 failed.
4. **Support IT governance**
 - Helps meet standards and regulations (e.g., coverage requirements). Since its measurable we can make quality as a number, ISO needs 90% of quality met for an app to be published.
5. **Assess quality and progress**
 - Provides info on **product quality, process effectiveness, and project status**.

Challenges:

- Partly manual → can be incomplete or error-prone.
- Main beneficiaries are often **other stakeholders**, so it may feel like extra effort without immediate benefit to the tester.

Vertical Vs Horizontal Traceability:

Vertical: Requirement → Component/code (implementation), it does not need a separate documentation. It just makes sure that what we have stated in the documentation is implemented. We can just link the java file to the requirement.

Horizontal: Requirement → All related test documentation & scripts. Horizontal traceability is about linking a requirement across all levels of test documentation and activities within the same testing process. That includes splitting a requirement into test conditions, then into test cases, and finally to test execution/results.

Test Oracle:

A test oracle is any source that provides the expected result of a test case—this could come from requirements, specifications, reference implementations, or other artifacts—so you can compare it to the actual result during testing and determine whether the test passes or fails.

Three probabilities for test oracles:

Tester: the tester derives the actual result from the logic of the code or the specifications, for example, add 2 numbers then the tester can easily drive that.

Prototype: creating a dynamic prototype can help derive the results.

Back-to-back testing (simplified)

1. What it is:

- The same software functionality is **developed independently by two or more teams**.

2. How it works:

- Give the **same input** to all versions.
- Compare the **outputs**.

3. Why:

- If the outputs are different → **there's a defect in one of the versions**.

4. Limitation:

- If all versions make the **same mistake**, the defect won't be detected.

Check the user manual sometimes the user manual tells the user what to expect therefore it tells the tester too.

If the feature was tested before you can use the older version output for testing to see if it still fails.

Sometimes you cannot expect the exact output so put an acceptable range

Experience testing is important for this one

Tester Vs Developer:

Developer: Develops solutions, and confirm that they work. (bias during confirmation)

Testers focus on validating the solution, not just building it.

They critically look for flaws rather than assuming everything works.

Curiosity and attention to detail help discover hidden defects.

Communication and empathy are important to report issues clearly and work with developers constructively.